Build-Systeme: Apache Maven

Carsten Gips (HSBI)

TL;DR
Zum Automatisieren von Arbeitsabläufen (Kompilieren, Testen, ...) stehen in der Java-Welt verschiedene Tools zur Verfügung: Apache Ant, Apache Maven und Gradle sind sicher die am bekanntesten darunter.
In Apache Maven ist bereits der typische Java-Standard-Lebenszyklus eingebaut und es müssen nur noch Abweichungen davon und Festlegung von Versionen und Dependencies in XML formuliert werden. Dies nennt man auch "*Convention over Configuration*".
Die Maven-Goals sind auswählbare Ziele und werden durch Plugins bereitgestellt. Zwischen den Goals sind Abhängigkeiten möglich (und bereits eingebaut). Über Properties kann man noch Namen und Versionsnummern o.ä. definieren.
Abhängigkeiten zu externen Bibliotheken werden als Dependencies formuliert: Am besten den Abschnitt von Maven-Central kopieren.
Videos

- VL Apache Maven
- Demo Maven-Projekt

---

1 Build-Tool Maven: Alternative zu Ant oder Gradle

```
mvn archetype:generate -DgroupId=de.hsbi.pm -DartifactId=my-project
                       -DarchetypeArtifactId=maven-archetype-quickstart
```

```
my-project
├── pom.xml
└── src
    ├── main
    │   └── java
    │       └── de
    │           └── fhb
    │               └── pm
    │                   └── App.java
    └── test
        └── java
            └── de
                └── fhb
                    └── pm
                        └── AppTest.java
```

Von der zeitlichen Entstehung her kommt Maven nach Ant, aber vor Gradle. Wie in Ant sind auch die Maven-Buildskripte XML-basierte Textdateien (Gradle nutzt eine Groovy-basierte DSL).

Allerdings hat Maven im Gegensatz zu Ant bereits ein Modell des Java-Entwicklungsprozess "eingebaut": Im Ant-Skript muss alles, was man tun möchte, explizit als Target formuliert werden, d.h. auch ein Kompilieren der Sourcen oder Ausführen der Tests muss extra als Target ins Ant-Skript geschrieben werden, um benutzbar zu sein. In Maven ist dieses Modell bereits implementiert, d.h. hier muss man lediglich zusätzliche oder abweichende Dinge im XML-File konfigurieren. Das nennt man auch *"convention over configuration"*.

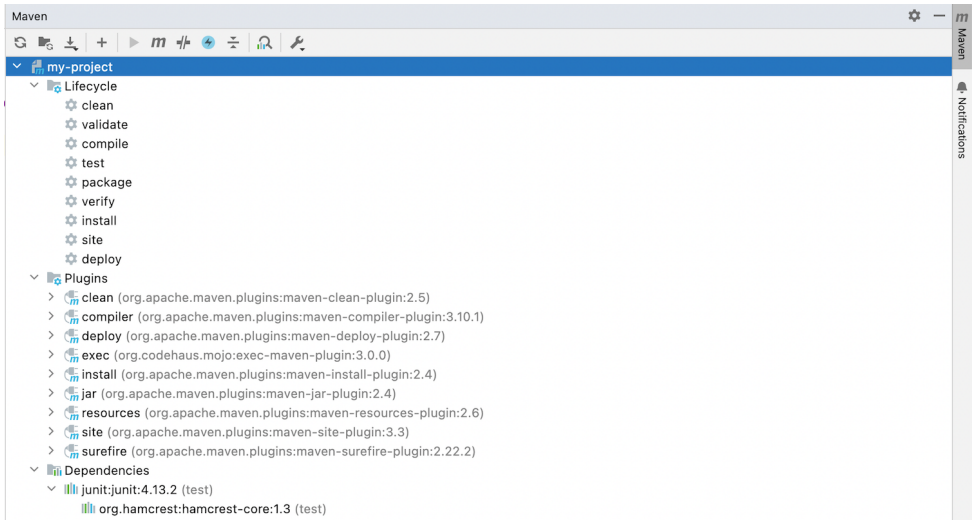
Der Maven-Aufruf `mvn archetype:generate -DgroupId=de.hsbi.pm -DartifactId=my-project -DarchetypeArtifactId=maven-archetype-quickstart` erzeugt mit Hilfe des Plugins

`archetype`, welches das Ziel (engl.: "*Maven goal*") `generate` bereitstellt, ein neues Projekt mit dem Namen `my-project` und der initialen Package-Struktur `de.hsbi.pm`. Das von Maven für die Projekterstellung genutzte Plugin ist unter der ID `maven-archetype-quickstart` in den Maven-Repositories (etwa Maven-Central) verfügbar, hier kann man mit der zusätzlichen Option `-DarchetypeVersion=1.4` auf die letzte Version schalten.

Die erzeugte Ordnerstruktur entspricht der Standardstruktur von Gradle (Gradle hat diese quasi von Maven übernommen). Die Konfigurationsdatei für Maven hat den Namen `pom.xml`.

Hinweis: Die `groupId` und `artifactId` werden auch für eine Veröffentlichung des Jar-Files des Projekts auf dem zentralen Maven-Repository Maven-Central genutzt. Von hier würde Maven auch als Abhängigkeit konfigurierte Bibliotheken herunterladen.

2 Lebenszyklus (eingebaut in Maven)



In Maven ist das typische Java-Entwicklungsmodell als "Lebenszyklus" implementiert.

Entsprechende Plugins stellen die jeweiligen "*Goals*" (Ziele) bereit. Dabei sind auch die Abhängigkeiten berücksichtigt, d.h. das Ziel `test` erfordert ein `compile` …

3 Project Object Model: *pom.xml*

```
<project>
    <!-- aktuelle Version für Maven 2.x-->
    <modelVersion>4.0.0</modelVersion>

    <!-- Basisinformationen -->
    <groupId>de.hsbi.pm</groupId>
    <artifactId>my-project</artifactId>
    <version>1.0-SNAPSHOT</version>
```

```
9
10      <!-- Eigenschaften, vergleichbar zu den Properties in Ant -->
11      <properties>
12          <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
13          <maven.compiler.release>17</maven.compiler.release>
14      </properties>
15
16      <!-- Abhängigkeiten zu externen Bibliotheken -->
17      <dependencies>
18          <dependency>
19              <groupId>junit</groupId>
20              <artifactId>junit</artifactId>
21              <version>4.13.2</version>
22              <scope>test</scope>
23          </dependency>
24      </dependencies>
25  </project>
```

Die Konfigurationsdatei `pom.xml` stellt die Konfiguration für das Maven-Projekt bereit ("Project Object Model").

Es werden mindestens der Name des Projekts sowie die Abhängigkeiten definiert.

Die `groupId` ist ein eindeutiger Bezeichner für die Organisation oder den Autor des Projekts. Oft wird hier einfach wie im obigen Beispiel eine Package-Struktur genutzt, aber wie im Fall von JUnit kann dies auch ein einfacher String (dort "`junit`") sein.

Die `artifactId` ist der eindeutige Name für das Projekt, d.h. unter diesem Namen wird das generierte Jar-File im Maven-Repository zu finden sein (sofern es denn veröffentlicht wird).

Über `dependencies` kann man benötigte Abhängigkeiten definieren, hier als Beispiel JUnit in der 4.x Variante … Diese werden bei Bedarf von Maven vom Maven-Repository heruntergeladen. Die Einträge für die Dependencies findet man ebenfalls auf MavenCentral.

4 Project Object Model: Plugins

```
1   <project>
2       ...
3
4       <!-- Plugins: Stellen eigene "Goals" zur Verfügung -->
5       <build>
6           <plugins>
7               <plugin>
8                   <!-- https://mvnrepository.com/artifact/org.apache.maven.
        plugins/maven-compiler-plugin -->
9                   <groupId>org.apache.maven.plugins</groupId>
10                  <artifactId>maven-compiler-plugin</artifactId>
11                  <version>3.10.1</version>
12              </plugin>
13
14              <plugin>
15                  <!-- https://mvnrepository.com/artifact/org.apache.maven.
        plugins/maven-surefire-plugin -->
```

```
16                  <groupId>org.apache.maven.plugins</groupId>
17                  <artifactId>maven-surefire-plugin</artifactId>
18                  <version>2.22.2</version>
19              </plugin>
20          </plugins>
21      </build>
22  </project>
```

Zusätzlich können die Phasen des Build-Prozesses konfiguriert werden, d.h. für die entsprechenden Plugins finden sich Abschnitte unter `<build><plugins>` in der `pom.xml`.

Auf maven.apache.org/plugins/index.html finden Sie eine Übersicht über häufig benutzte Plugins sowie die von den Plugins bereitgestellten Goals sowie Konfigurationsmöglichkeiten.

Die entsprechenden POM-Einträge finden Sie analog zu den Dependencies ebenfalls auf MavenCentral (Tag "plugin" statt "dependency").

Plugins können aber auch selbst erstellt werden und in das Projekt eingebunden werden, ein erster Einstieg ist die Plugin-API.

5 Und wie lasse ich jetzt eine Anwendung mal laufen?

- `mvn clean`: Lösche alle generierten Artefakte, beispielsweise `.class`-Dateien.
- `mvn compile` => `mvn compiler:compile`: Übersetze die Sourcen und schiebe die generierten `.class`-Dateien in den Ordner `target/classes/` (Default). Dazu werden alle Abhängigkeiten aufgelöst und bei Bedarf (neu) heruntergeladen (Default: Userverzeichnis, Ordner `.m2/`).
- `mvn test` => `mvn surefire:test`: Lasse die Tests laufen. Hängt von `compile` ab. Namenskonvention: Alle Klassen mit `*Test.java` und `Test*.java` im Standard-Testordner `src/test/java/` werden betrachtet (und weitere, vgl. maven.apache.org/surefire/maven-surefire-plugin/examples/junit-platform.html).
- `mvn package`: Hängt von `compile` ab und erzeugt ein Jar-File mit dem Namen "artifactId-version.jar" im Ordner `target/`. Mit `mvn install` kann man dieses Jar-File dann auch dem lokalen Repository im Home-Verzeichnis des Users (`.m2/`) hinzufügen.
- `mvn exec:java -Dexec.mainClass="de.hsbi.pm.Main"`: Hängt von `compile` ab und führt die Klasse `de.hsbi.pm.Main` aus.

Demo: pom.xml

6 Wrap-Up

Apache Maven: maven.apache.org, Maven Getting Started Guide

- Automatisieren von Arbeitabläufen
- Apache Maven: Goals, Properties, Dependencies => "*Convention over Configuration*", Java-Standard-Lebenszyklus eingebaut

  - Goals sind auswählbare Ziele, bereitgestellt durch Plugins
  - Abhängigkeiten zwischen Goals möglich
  - Properties agieren wie Variablen, etwa für Versionsnummern
  - Abhängigkeiten zu externen Bibliotheken werden als Dependencies formuliert: Abschnitt von Maven-Central kopieren

7 Zum Nachlesen

- Ullenboom (2021)
- Inden (2013)

Lernziele

- k3: Schreiben einfacher Maven-Skripte zu Übersetzen des Projekts, zum Testen und zum Erzeugen von Jar-Files
- k3: Nutzung von Maven-Properties
- k3: Einbinden externer Bibliotheken als Dependencies
- k3: Ausführen von Maven-Goals aus IDE heraus und Einbindung als Builder

Quizzes

- Quiz Apache Maven (ILIAS)

Quellen

Inden, M. 2013. *Der Weg zum Java-Profi*. 2. Aufl. dpunkt.verlag.

Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 9d7ab85 (Remove Hugo: remove pandoc escaping on video links, 2025-04-30)